SAECOIN — THREAT MODEL

Document: Threat_Model.md
Project: SAECOIN
Version: 1.0
Status: Draft — Security Review Required
Consensus: Proof of Work (PoW)
Transaction Model: UTXO
Target Platform: Windows 11, Android, Linux
Development PC Baseline: 16 GB RAM

---

1. PURPOSE

This document identifies security threats that may affect the SAECOIN blockchain, nodes, miners, wallets, APIs, development infrastructure, and users.

The objectives are to:

- Identify critical assets.
- Identify possible attackers and their capabilities.
- Analyze blockchain-specific attacks.
- Define security controls and mitigations.
- Establish detection and incident response procedures.
- Define security requirements before testnet and mainnet launch.

This document is a threat-analysis plan. It does not certify that SAECOIN is secure.

---

2. SECURITY OBJECTIVES

SAECOIN must aim to maintain the following properties:

2.1 Integrity

- Transactions must follow protocol rules.
- Blocks must pass consensus validation.
- No unauthorized coins may be created.
- UTXO state must remain consistent across nodes.

2.2 Availability

- Nodes should remain operational during network attacks.
- P2P communication should recover from peer failures.
- Wallets should remain usable when individual nodes become unavailable.

2.3 Confidentiality

- Private keys and seed phrases must remain secret.
- Wallet backups must be protected.
- Sensitive user information must not be unnecessarily exposed.

2.4 Authenticity

- Software releases must be verifiable.
- Transactions must be cryptographically authorized.
- Network messages must be validated before processing.

2.5 Monetary Integrity

- Block rewards must follow the consensus schedule.
- Transaction fees must be calculated correctly.
- Supply limits must be enforced by every validating node.

---

3. CRITICAL ASSETS

Asset| Importance| Security Requirement
Private keys| Critical| Must never be exposed
Seed phrases| Critical| Secure generation and storage
Consensus rules| Critical| Identical validation across nodes
UTXO database| Critical| Integrity and recoverability
Block history| Critical| Valid chain verification
Mining infrastructure| High| Availability and integrity
P2P network| High| Resistance to malicious peers
Wallet software| Critical| Secure transaction signing
Release signing keys| Critical| Restricted access
Source code repository| High| Access control and review
API/RPC services| High| Authentication and access limits
Explorer data| Medium| Accurate and verifiable information
User privacy| High| Data minimization

---

4. THREAT ACTORS

4.1 Malicious Miners

Possible capabilities:

- Attempting double-spend attacks.
- Producing competing chains.
- Manipulating block timestamps.

- Attempting selfish mining.
- Withholding discovered blocks.

4.2 Majority Hashpower Attacker

An attacker controlling a substantial portion of the network's mining power may attempt:

- Reorganizing recent blocks.
- Reversing their own recent payments.
- Preventing selected transactions from confirming.
- Censoring transactions.

A majority hashpower attack does not automatically allow an attacker to steal arbitrary coins from another user's wallet or create valid coins outside consensus rules.

4.3 Malicious P2P Peers

Possible capabilities:

- Sending malformed messages.
- Advertising invalid blocks.
- Flooding nodes with requests.
- Attempting eclipse or Sybil attacks.
- Exploiting parser vulnerabilities.

4.4 Wallet Attackers

Possible capabilities:

- Stealing seed phrases.
- Distributing fake wallet applications.
- Replacing copied addresses.
- Using malware to access wallet files.
- Tricking users into signing malicious transactions.

4.5 Supply-Chain Attackers

Possible capabilities:

- Compromising dependencies.
- Publishing malicious packages.
- Modifying build tools.
- Compromising GitHub accounts.
- Replacing official release binaries.

4.6 Infrastructure Attackers

Possible capabilities:

- DDoS attacks.
- Server compromise.
- Database manipulation.
- Unauthorized API access.
- Theft of release credentials.

4.7 Accidental Failures

Not all threats involve malicious actors.

Examples:

- Consensus implementation bugs.
- Integer overflow.
- Incorrect difficulty calculations.
- Database corruption.
- Software upgrade incompatibility.
- Accidental private-key deletion.

---

5. BLOCKCHAIN AND CONSENSUS THREATS

5.1 Majority Hashpower Attack

Risk: Critical

An attacker controlling a majority of effective mining power may reorganize the chain and reverse their own recent transactions.

Mitigations:

- Encourage independent miners.
- Avoid excessive mining-pool concentration.
- Monitor hashpower distribution.
- Monitor unusually deep chain reorganizations.
- Require additional confirmations for high-value payments.
- Develop an emergency response procedure.

Residual Risk: A sufficiently powerful attacker may still reorganize the chain.

5.2 Double-Spend Attack

Risk: High

An attacker may attempt to spend the same UTXO in competing transactions.

Mitigations:

- Strict UTXO validation.
- Reject transactions spending already-spent outputs.
- Enforce canonical chain selection rules.
- Provide confirmation-depth information to wallets and merchants.
- Avoid treating unconfirmed transactions as final payments.

Six confirmations may be used as an initial payment policy, but this is not absolute finality. The appropriate confirmation policy must be evaluated against network security and transaction value.

5.3 Selfish Mining

Risk: Medium to High

A miner may withhold discovered blocks and strategically release them to gain an advantage.

Mitigations:

- Analyze block propagation behavior.
- Simulate competing mining strategies.
- Monitor unusual orphan and stale-block rates.
- Encourage geographically and operationally distributed nodes.

5.4 Block Withholding

Risk: Medium

A miner participating in a mining pool may withhold valid blocks or proofs to damage the pool or manipulate rewards.

Mitigations:

- Design verifiable pool accounting if pool mining is introduced.
- Keep mining reward calculations transparent.
- Monitor abnormal share-to-block behavior.

Pool mining is not required for the initial solo-mining implementation.

5.5 Timestamp Manipulation

Risk: High

Malicious miners may manipulate block timestamps to influence difficulty calculations or chain behavior.

Mitigations:

- Define strict timestamp validation rules.
- Define acceptable future-time limits.
- Use a deterministic difficulty algorithm.
- Test timestamp edge cases.
- Prevent timestamp rules from creating inconsistent validation between nodes.

The exact timestamp rules remain an open protocol decision.

5.6 Difficulty Manipulation

Risk: Critical

Incorrect difficulty adjustment may allow unusually fast mining, delayed blocks, or unstable network operation.

Mitigations:

- Select and document a deterministic difficulty algorithm.
- Test sudden hashpower increases and decreases.
- Simulate long periods of unstable mining.
- Use property-based testing.
- Test boundary conditions and integer calculations.

The difficulty formula must be finalized and independently reviewed before mainnet.

5.7 Deep Chain Reorganization

Risk: Critical

A chain reorganization may replace a significant portion of previously accepted blocks.

Mitigations:

- Monitor reorganization depth.
- Alert users and services about unusual reorganizations.
- Keep reliable block and UTXO recovery mechanisms.
- Establish a documented emergency procedure.
- Test wallet and explorer behavior during reorganizations.

Do not assume that a particular confirmation count makes a transaction irreversible.

5.8 Invalid Block and Unauthorized Inflation

Risk: Critical

A software defect may allow an invalid block reward or unauthorized coin creation.

Mitigations:

- Validate every block reward.
- Enforce the subsidy schedule in consensus code.
- Use checked integer arithmetic.
- Reject invalid coinbase transactions.
- Test total supply calculations independently.
- Verify that every node applies identical rules.

The proposed 21,000,000 SAC supply and reward schedule must undergo mathematical verification before finalization.

---

6. NETWORK AND P2P THREATS

6.1 Sybil Attack

Risk: High

An attacker may operate many network identities to influence peer discovery or isolate nodes.

Mitigations:

- Maintain diverse peer connections.
- Limit excessive connections from related network sources.
- Use peer scoring and reputation carefully.
- Avoid trusting a single peer.
- Provide reliable bootstrap discovery mechanisms.

6.2 Eclipse Attack

Risk: High

An attacker may surround a node with malicious peers and isolate it from the honest network.

Mitigations:

- Maintain diverse outbound peers.
- Avoid relying on one network provider.
- Rotate peer connections.
- Validate all received blocks and transactions.
- Monitor unusual peer concentration.

6.3 Denial of Service (DoS)

Risk: High

An attacker may exhaust CPU, RAM, bandwidth, disk space, or connection capacity.

Mitigations:

- Set message-size limits.
- Limit concurrent connections.
- Apply request timeouts.
- Rate-limit expensive operations.
- Reject malformed messages early.
- Avoid unbounded memory allocation.
- Monitor resource usage.

6.4 Malformed P2P Messages

Risk: Critical

Malformed network messages may trigger crashes, memory errors, or inconsistent node behavior.

Mitigations:

- Use safe parsing.
- Validate message lengths.
- Reject unexpected fields.
- Apply strict input limits.
- Fuzz all network parsers.
- Never trust peer-provided data.

6.5 Transaction Spam

Risk: Medium to High

Attackers may flood the network with excessive low-value transactions.

Mitigations:

- Define transaction-size limits.
- Establish fee and relay policies.
- Limit memory-pool resource consumption.
- Apply per-peer request limits.
- Monitor transaction volume.

The exact fee policy and transaction limits remain to be finalized.

---

7. TRANSACTION AND CRYPTOGRAPHIC THREATS

7.1 Replay Attack

Risk: High

A transaction intended for one network may be replayed on another network if transaction signing does not distinguish networks.

Mitigations:

- Include a network-specific identifier in transaction signing.
- Define separate network identifiers for mainnet and testnet.
- Validate network-specific transaction rules.

7.2 Transaction Malleability

Risk: Medium to High

Different representations of a transaction may create problems for transaction identification or dependent transactions.

Mitigations:

- Use canonical transaction serialization.
- Define transaction identifier calculation precisely.
- Select and review appropriate signature encoding.
- Test alternative and invalid encodings.

7.3 Weak Randomness

Risk: Critical

Weak random-number generation may expose private keys.

Mitigations:

- Use operating-system cryptographically secure randomness.
- Never generate wallet keys using ordinary random functions.
- Test key-generation error handling.
- Never hardcode private keys or seeds.

7.4 Signature Verification Failure

Risk: Critical

Incorrect signature verification may allow unauthorized spending or reject valid transactions.

Mitigations:

- Select a well-reviewed cryptographic library.
- Use established cryptographic standards.
- Test valid and invalid signatures.
- Include edge cases in test vectors.
- Obtain independent cryptographic review.

The signature algorithm has not yet been finalized.

7.5 Integer Overflow and Precision Errors

Risk: Critical

Incorrect arithmetic may cause unauthorized money creation, incorrect fees, or invalid balances.

Mitigations:

- Store amounts as integer smallest units.
- Never use floating-point arithmetic for consensus money calculations.
- Use checked addition, subtraction, and multiplication.
- Reject values outside permitted ranges.
- Test maximum-value and boundary cases.

---

8. WALLET SECURITY THREATS

8.1 Seed Phrase Theft

Risk: Critical

A stolen seed phrase may allow complete control of the associated wallet funds.

Mitigations:

- Generate seeds using secure randomness.
- Display seed phrases only when necessary.
- Warn users never to share seed phrases.
- Avoid cloud synchronization of unencrypted secrets.
- Provide secure backup instructions.

8.2 Wallet File Theft

Risk: Critical

An attacker may copy an encrypted wallet file and attempt offline password guessing.

Mitigations:

- Use a reviewed key-derivation function.
- Use authenticated encryption.
- Apply appropriate password-hardening parameters.
- Never store plaintext private keys unnecessarily.
- Test backup and recovery procedures.

The wallet encryption standard must be selected before implementation.

8.3 Password Brute Force

Risk: High

Weak passwords may allow an attacker to decrypt stolen wallet files.

Mitigations:

- Recommend strong passwords.
- Use a suitable memory-hard key-derivation function.
- Avoid weak default passwords.
- Explain the consequences of password loss.

8.4 Clipboard Address Substitution

Risk: High

Malware may replace a copied payment address with an attacker-controlled address.

Mitigations:

- Show the complete recipient address before signing.
- Encourage users to verify the first and last characters.
- Provide a clear transaction confirmation screen.
- Never sign transactions silently.

8.5 Fake Wallet Applications

Risk: Critical

Attackers may distribute fake SAECOIN wallet software to steal funds.

Mitigations:

- Publish releases through official channels.
- Sign release packages.

- Publish cryptographic checksums.
- Provide clear verification instructions.
- Warn users against unofficial wallet downloads.

8.6 Device Loss

Risk: High

A lost or damaged device may make wallet access impossible.

Mitigations:

- Provide a secure recovery process.
- Explain seed backup clearly.
- Test recovery on a separate device.
- Warn users that losing both device and backup may result in permanent loss.

---

9. SOFTWARE SUPPLY-CHAIN THREATS

9.1 Malicious Dependency

Risk: High

A compromised Rust crate or frontend package may introduce malicious code.

Mitigations:

- Review dependencies before adoption.
- Commit dependency lockfiles.
- Avoid unnecessary dependencies.
- Monitor security advisories.
- Update dependencies through reviewed changes.
- Audit critical cryptographic dependencies.

9.2 Compromised Build Environment

Risk: High

An attacker may modify build tools or development environments.

Mitigations:

- Use trusted toolchain versions.
- Document reproducible build steps.
- Keep development systems updated.
- Restrict administrative access.

- Verify build artifacts independently.

9.3 Release Signing Key Compromise

Risk: Critical

A compromised release signing key may allow attackers to distribute fake official software.

Mitigations:

- Store signing keys securely.
- Restrict access to authorized maintainers.
- Use hardware-backed protection where practical.
- Keep offline recovery procedures.
- Establish a signing-key rotation plan.

9.4 Unauthorized Source Code Changes

Risk: High

An attacker may attempt to modify consensus or wallet code.

Mitigations:

- Require code review.
- Protect main branches.
- Require automated tests.
- Restrict repository permissions.
- Review all consensus-related changes carefully.

---

10. DATABASE AND STORAGE THREATS

10.1 Database Corruption

Risk: High

A node database may become corrupted because of crashes, disk failures, or software defects.

Mitigations:

- Use transactional database operations where appropriate.
- Validate recovered state.
- Maintain tested backup procedures.
- Support chain resynchronization.
- Test unexpected shutdown scenarios.

RocksDB is a provisional storage choice and must be benchmarked and reviewed.

10.2 Disk Exhaustion

Risk: High

Blockchain data may consume all available disk space.

Mitigations:

- Monitor disk usage.
- Provide storage estimates.
- Warn before disk exhaustion.
- Define pruning policies if supported.
- Handle low-disk conditions safely.

10.3 Corrupted UTXO State

Risk: Critical

Incorrect UTXO updates may create invalid balances or allow repeated spending.

Mitigations:

- Validate state transitions.
- Use atomic database updates.
- Test rollback and reorganization behavior.
- Compare state against independently validated block history.

---

11. API AND EXPLORER THREATS

11.1 Unauthorized RPC Access

Risk: Critical

An exposed administrative RPC interface may allow attackers to control node operations.

Mitigations:

- Bind sensitive RPC interfaces to localhost by default.
- Require authentication for remote administrative access.
- Use network restrictions.
- Disable unnecessary methods.
- Never expose private-key operations through public APIs.

11.2 API Abuse

Risk: Medium to High

Attackers may overload public API services.

Mitigations:

- Apply rate limits.
- Use request-size limits.
- Set timeouts.
- Monitor unusual request patterns.
- Separate public read-only services from administrative operations.

11.3 Explorer Data Manipulation

Risk: Medium

A compromised or malfunctioning explorer may display incorrect balances, transactions, or confirmation counts.

Mitigations:

- Derive explorer data from validated node information.
- Display block height and confirmation status.
- Provide transaction identifiers.
- Avoid presenting explorer data as an independent source of consensus truth.

---

12. PRIVACY THREATS

Potential privacy risks include:

- IP address exposure.
- Transaction graph analysis.
- Wallet address reuse.
- Public transaction history correlation.
- Excessive API logging.

Mitigations:

- Minimize unnecessary personal data collection.
- Avoid storing sensitive information without a clear need.
- Provide clear privacy documentation.
- Do not promise transaction anonymity unless a specific privacy protocol has been designed and independently reviewed.

SAECOIN must not claim to be an anonymous cryptocurrency merely because wallet addresses do not contain real names.

---

13. RISK PRIORITIZATION

Threat| Severity| Priority
Consensus implementation bug| Critical| P0
Unauthorized coin creation| Critical| P0
Private-key theft| Critical| P0
Majority hashpower attack| Critical| P0
Signature verification failure| Critical| P0
Integer overflow| Critical| P0
Invalid block acceptance| Critical| P0
Release signing compromise| Critical| P0
Malformed P2P message| Critical| P0
Database corruption| High| P1
Double spending| High| P1
Eclipse attack| High| P1
Wallet malware| High| P1
Dependency compromise| High| P1
API abuse| Medium–High| P2
Explorer misinformation| Medium| P2

Priority definitions:

- P0: Must be addressed before mainnet.
- P1: Must be addressed before public production use.
- P2: Must have a documented mitigation and monitoring plan.

Severity and priority are initial assessments. They must be reviewed as the protocol and implementation evolve.

---

14. SECURITY TESTING REQUIREMENTS

14.1 Unit Testing

Test:

- Transaction validation.
- Signature verification.
- Block reward calculation.
- Difficulty calculations.
- UTXO updates.
- Fee calculations.

- Serialization and deserialization.

14.2 Integration Testing

Test:

- Multiple nodes communicating.
- Transaction propagation.
- Block propagation.
- Chain synchronization.
- Reorganization handling.
- Wallet-to-node communication.

14.3 Fuzz Testing

Fuzz:

- P2P message parsers.
- Transaction parsers.
- Block parsers.
- Serialization functions.
- Wallet import functions.

14.4 Property-Based Testing

Verify properties such as:

- No UTXO can be spent twice.
- Invalid signatures are rejected.
- Invalid blocks are rejected.
- Total issued supply never exceeds the protocol limit.
- All valid nodes calculate identical state transitions.

14.5 Attack Simulation

Simulate:

- High hashpower concentration.
- Sudden miner departures.
- Network partitions.
- Malicious peers.
- Transaction flooding.
- Reorganization events.
- Timestamp manipulation.

14.6 Independent Security Review

Before mainnet:

- Review consensus implementation.
- Review cryptographic choices.
- Review wallet key management.
- Review network protocol.
- Review build and release procedures.
- Document and resolve critical findings.

---

15. MONITORING AND DETECTION

Node operators should monitor:

- Block production intervals.
- Network hash rate estimates.
- Difficulty changes.
- Reorganization depth.
- Invalid block frequency.
- Peer connection patterns.
- Memory usage.
- CPU usage.
- Disk usage.
- Network traffic.
- Node crashes.
- Unusual transaction volume.

Security alerts should be generated for unusual conditions.

Monitoring data must not expose wallet private keys or seed phrases.

---

16. INCIDENT RESPONSE PLAN

If a serious vulnerability or attack is detected, follow this procedure.

Step 1 — Identify

- Confirm the reported issue.
- Determine affected software versions.
- Determine whether consensus or user funds are at risk.

Step 2 — Contain

- Disable affected non-essential services if necessary.
- Restrict compromised infrastructure.
- Protect signing keys and credentials.

- Communicate clearly with node operators.

Step 3 — Investigate

- Preserve relevant logs.
- Reproduce the issue in a controlled environment.
- Determine the root cause.
- Assess possible financial impact.

Step 4 — Fix

- Develop a reviewed patch.
- Add regression tests.
- Verify compatibility.
- Obtain independent review for critical consensus changes.

Step 5 — Release

- Sign the release.
- Publish checksums.
- Publish security notices.
- Provide upgrade instructions.

Step 6 — Review

- Document the incident.
- Identify failed controls.
- Update this threat model.
- Add tests to prevent recurrence.

Consensus emergencies require a carefully coordinated process. A rushed protocol change may create additional chain-splitting or security risks.

---

17. RESPONSIBLE DISCLOSURE

SAECOIN should establish a security contact before public launch.

Security researchers should be encouraged to report vulnerabilities privately before publishing technical details that could expose users or network infrastructure.

A future disclosure policy should define:

- Official security contact.
- Reporting format.
- Expected response time.
- Vulnerability severity categories.

- Coordinated disclosure expectations.
- Reward policy, if any.

No bug bounty program is assumed to exist at this stage.

---

18. MAINNET SECURITY NO-GO CONDITIONS

Mainnet must not launch if any of the following conditions remain unresolved:

1. Consensus rules are not fully specified.
2. Block reward and supply calculations are not mathematically verified.
3. Critical consensus tests are failing.
4. Invalid blocks can be accepted.
5. Private keys can be exposed through normal wallet operation.
6. Signature verification has not been independently reviewed.
7. Integer overflow risks remain in consensus calculations.
8. Network parsers have unresolved critical vulnerabilities.
9. Genesis block parameters are not reproducible.
10. Release binaries cannot be authenticated.
11. Backup and wallet recovery procedures have not been tested.
12. Critical security review findings remain unresolved.
13. Emergency response procedures are not documented.

---

19. THREAT MODEL LIMITATIONS

This document cannot guarantee protection against every possible attack.

The following limitations apply:

- New vulnerabilities may be discovered.
- Cryptographic assumptions may change.
- Mining power may become concentrated.
- Network conditions may change.
- Third-party dependencies may become compromised.
- Human error may still cause financial loss.
- A security review cannot prove the absence of all defects.

Risk assessments must be updated as implementation and testing progress.

---

20. REVIEW SCHEDULE

This document should be reviewed:

- Before starting consensus implementation.
- Before launching the first public testnet.
- Before mainnet launch.
- Before major consensus upgrades.
- After any critical security incident.
- At least once every six months during active development.

---

21. FINAL SECURITY PRINCIPLE

SAECOIN must prioritize correctness and security over development speed.

No feature should be considered complete merely because it works in a normal demonstration.

Every critical component must be tested against invalid inputs, malicious behavior, unexpected failures, and recovery scenarios.

Mainnet readiness requires documented evidence, independent review, and successful testing — not assumptions.

END OF DOCUMENT